

Github: <https://github.com/David2DAM/CompletaTarea4DavidDAD.git>

Primero hago todos los imports:

```
PS C:\Users\alumnadoM\Desktop\proyectoDRCDAD\Act4David> npm install react-router-dom

PS C:\Users\alumnadoM\Desktop\proyectoDRCDAD\Act4David> npm install @emotion/styled

found 0 vulnerabilities
PS C:\Users\alumnadoM\Desktop\proyectoDRCDAD\Act4David> npm install @emotion/react

found 0 vulnerabilities
PS C:\Users\alumnadoM\Desktop\proyectoDRCDAD\Act4David> npm install @mui/icons-material

found 0 vulnerabilities
PS C:\Users\alumnadoM\Desktop\proyectoDRCDAD\Act4David> npm install @mui/material

found 0 vulnerabilities
PS C:\Users\alumnadoM\Desktop\proyectoDRCDAD\Act4David> npm install react-redux

found 0 vulnerabilities
PS C:\Users\alumnadoM\Desktop\proyectoDRCDAD\Act4David> npm install @reduxjs/toolkit
```

(Esto es mas un apunte para mi que otra cosa)

Conexión del login con la base de datos

Con lo que se explicó en clase sobre el backend y los endpoints, reemplaza tu comprobación de usuario y login correcto con la llamada al endpoint /login para conectarte a la base de datos y hacer la autenticación del usuario.

Creación del menú común a todas las páginas

El menú será común a todas las páginas de la aplicación: tanto si estamos en Inicio (en /home) como si estamos en Informes (/reports) queremos ver el mismo menú. Por lo tanto lo vamos a programar en un fichero que estará dentro de la carpeta components: en el frontend/src/components/Menu.tsx.

Tendrás que trasladar la parte de la navegación entre páginas (useNavigate, useDispatch, useSelector) a esa parte del código, puesto que será común en todas las páginas

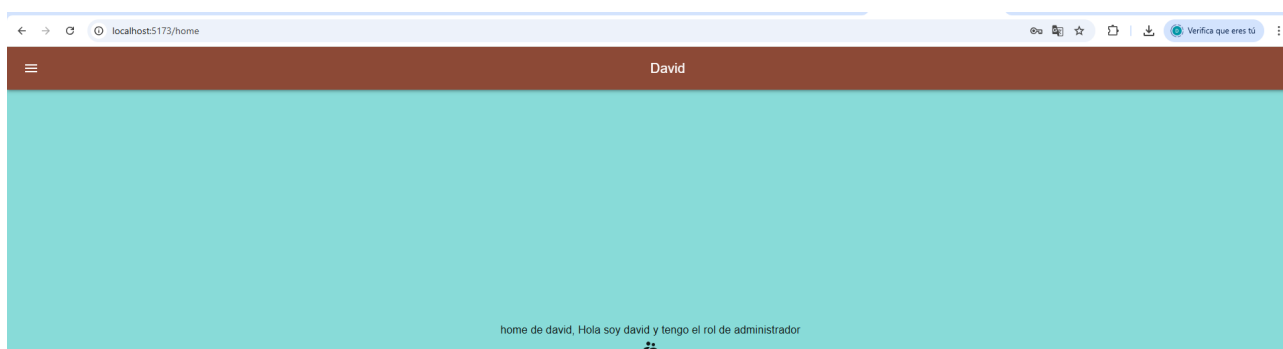
El menú debe ser responsive y debe aparecer el nombre del usuario que está logueado junto con un icono representativo del rol del usuario. Necesitarás usar, entre otros, los componentes AppBar para la barra superior (<https://mui.com/material-ui/react-app-bar/>)

```

<AppBar position="fixed" sx={{ width: '100%', boxShadow: 3 }}>
  <Toolbar>
    <IconButton
      size="large"
      edge="start"
      color="inherit"
      aria-label="menu"
      sx={{ mr: 2 }}
    >
      <MenuIcon />
    </IconButton>
    <Typography variant="h6" component="div" sx={{ flexGrow: 1 }}>
      David
    </Typography>
  </Toolbar>
</AppBar>

```

Este es el código de mi topBar, en mui por lo menos o no vi un componente topbar así que es un appBar con position fixed que se queda arriba quieto, además es width 100% para que se extienda completamente y además se levanta un poco lo que le da un poco de relieve a la página (Por si hay dudas es la appBar del link de muy pero adaptada a mis necesidades de posicionamiento)



Así queda

y Drawer que es lo que se abre cuando picas el botón hamburguesa (<https://mui.com/material-ui/react-drawer/>). Ayúdame de la documentación de MUI, hay infinidad de ejemplos. Tienes libertad para usar otros tipos de menús, siempre y cuando seas capaz de modificar el código sin problema

```

const [open, setOpen] = React.useState(false);
//Este es el estado del drawer, por defecto cerrado

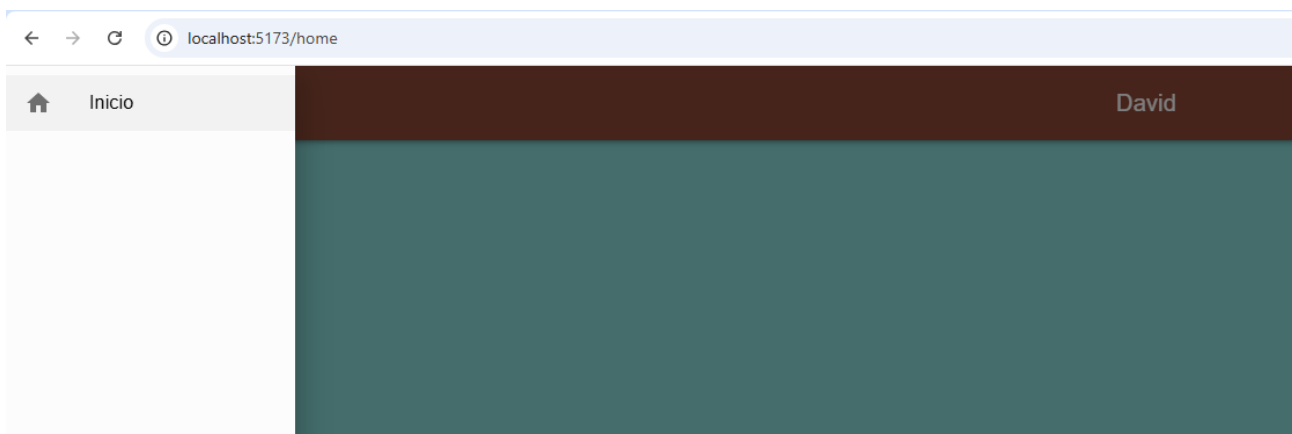
const toggleDrawer = (newOpen: boolean) => () => {
  setOpen(newOpen);
  //Lo que hace aquí es que cambia el estado del drawer
};
//Esta función es la que cambia el estado del drawer de la librería de mui de drawer, no me pregunten por que usa dos lambdas https://mui.com/material-ui/react-drawer/

const DrawerList = (

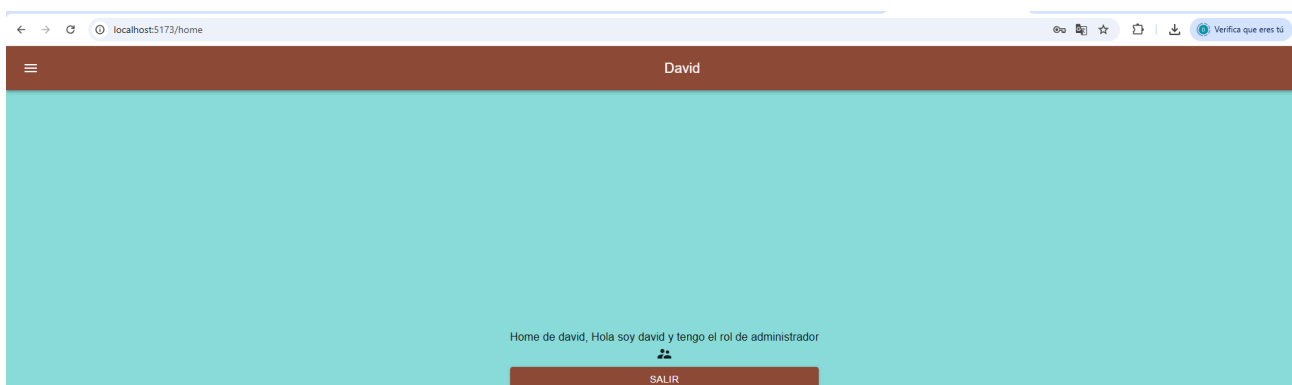
```

Creo que para esta parte de la tarea solo necesitaria esto como para demostrar que lo he hecho, pero yo esta parte la vine a hacer mas tarde por que no lei bien, entonces tengo el drawer con un item hecho como se indica mas adelante

```
const DrawerList = (  
  <Box sx={{ width: 250 }} role="presentation" onClick={toggleDrawer(false)}>  
    <List>  
      <Link to={'/home'} style={{ textDecoration: 'none', color: 'black' }}>  
        <ListItem disablePadding>  
          <ListItemButton>  
            <ListItemIcon>  
              {<HomeIcon />}  
            </ListItemIcon>  
            <ListItemText primary="Inicio" />  
          </ListItemButton>  
        </ListItem>  
      </Link>  
    </List>  
  </Box>  
>;  
//Aquí vamos a añadir nuestros items con sus iconos y un textito de a donde llevar
```



Y aquí abro el drawer, el boton todavia no tiene funcionalidad pero ahi esta to majo

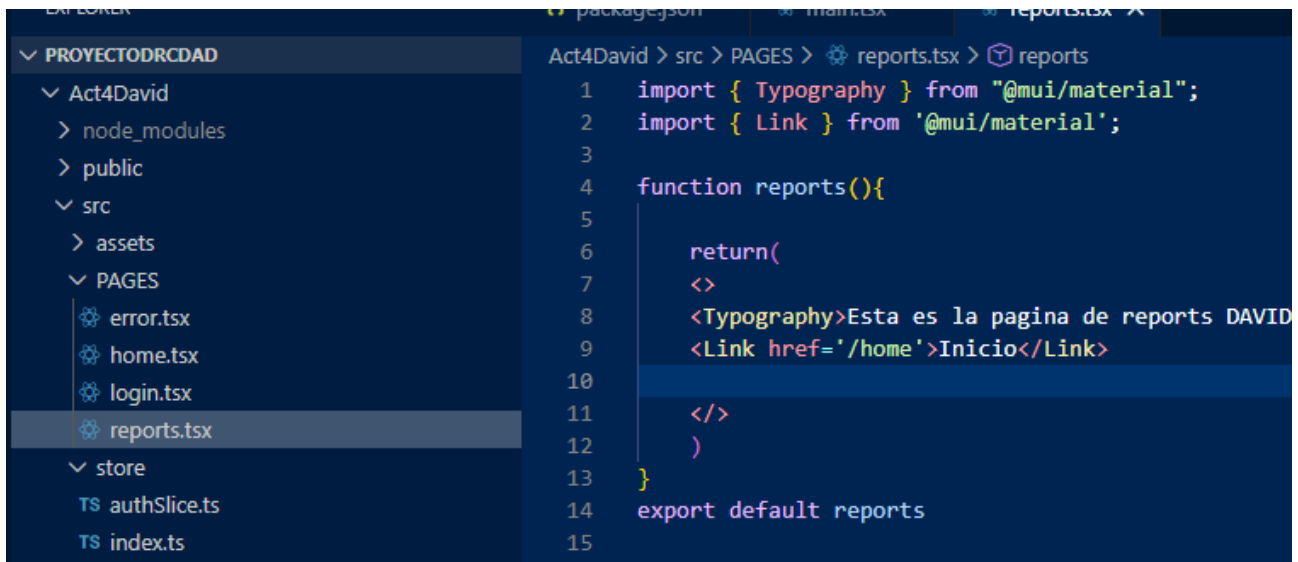


Y asi quedo

Lo único que debes tener en cuenta aquí es que para navegar entre las diferentes páginas (por ahora Inicio y Informes) cuando piques en el menú tienes que usar el componente de la librería react-router-dom. Este componente debe envolver al componente donde vayas a hacer la navegación. Te pongo dos ejemplos:

Ejemplo 1

Si quieres poner un link en alguna parte del código que te lleve a la página <http://localhost:5173/home>, sería así:

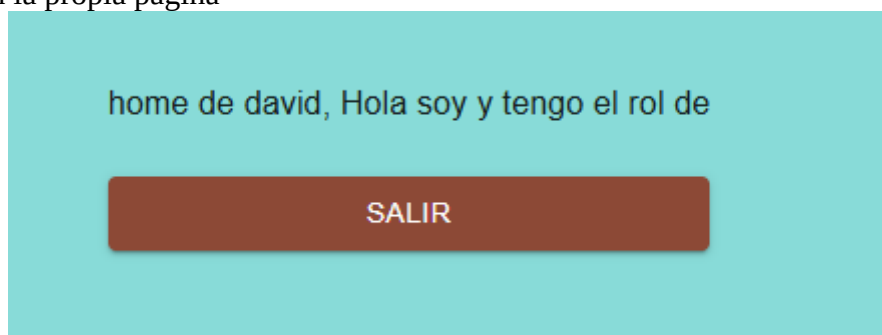


```
1 import { Typography } from "@mui/material";
2 import { Link } from '@mui/material';
3
4 function reports(){
5
6     return(
7         <>
8         <Typography>Esta es la pagina de reports DAVID
9         <Link href='/home'>Inicio</Link>
10
11         </>
12     )
13 }
14 export default reports
15
```

Para probarlo lo añadi a reports



Y aquí se ve en la propia pagina



Cuando entro se ve que entre atraves de este link por que el userState no tiene datos y no printea nada

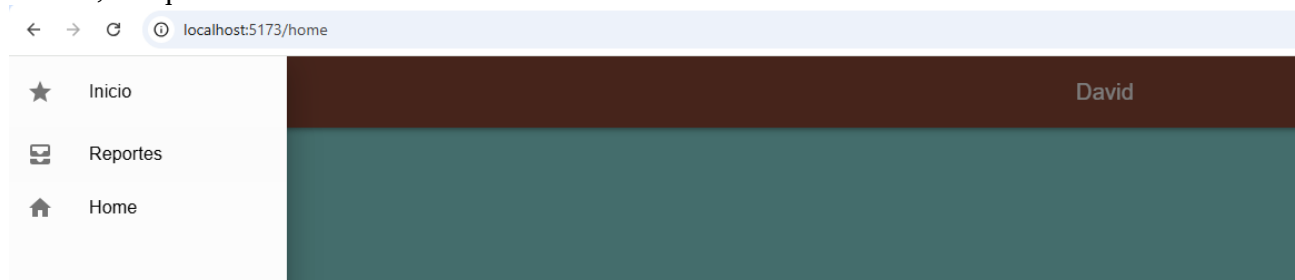
En este caso te aparecerá en el código la palabra Inicio que será un enlace hay la página /home. Es decir, lo que va dentro de link to="/<pagina>" Trozo de código que nos redirigirá a la página requerida cuando piquemos sobre el link

Ejemplo 2

En este ejemplo vamos a envolver tanto un botón de icono como un texto para que ambos me lleven a la página requerida cuando pique en ellos. Para verlo de forma más concreta: cuando se despliegue la lista del componente drawer quiero que al picar bien en el icono de la casa como en la palabra Inicio se vaya a la página <http://localhost:5173/home>

```
const DrawerList = (  
  <Box sx={{ width: 250 }} role="presentation" onClick={toggleDrawer(false)}>  
    <List>  
      <Link to={'/' } style={{ textDecoration: 'none', color: 'black' }}>  
        <ListItem disablePadding>  
          <ListItemButton>  
            <ListItemIcon>  
              {<HomeIcon />}  
            </ListItemIcon>  
            <ListItemText primary="Inicio" />  
          </ListItemButton>  
        </ListItem>  
      </Link>  
    </List>  
  </Box>  
>);
```

Con este formato mi unico boton me manda al login, quiero añadir un boton a cada pagina en mi drawer, así que...



Lo primero así me quedo con iconos y links funcionales



Esto lo hago solamente añadiendo un bloque igual al primero pero cambiando icono, link y texto nuevo por pagina

```

    /* Quiero añadir un boton a cada pagina, lo hago añadiendo otro bloque directamente*/
    <Link to="/reports" style={{ textDecoration: "none", color: "black" }}>
      <ListItem disablePadding>
        <ListItemButton>
          <ListItemIcon>
            <AllInboxIcon />
          </ListItemIcon>
          <ListItemText primary="Reportes" />
        </ListItemButton>
      </ListItem>
    </Link>

    /* Añado un ultimo icono, como el drawer es comun a las 3 paginas por un tema de programacion eficiente vas a poder ir a tu misma pagina desde la misma lo cual no
    debería ser un problema segun yo*/
    <Link to="/Home" style={{ textDecoration: "none", color: "black" }}>
      <ListItem disablePadding>
        <ListItemButton>
          <ListItemIcon>
            <HomeIcon />
          </ListItemIcon>
          <ListItemText primary="Home" />
        </ListItemButton>
      </ListItem>
    </Link>
  </Box>

```

4. Cómo evitar navegar a /home sin estar logueado: hook useEffect()

Hasta ahora si ponemos en la URL <http://localhost:5173/home> el enrutador nos montará el componente <Home>, que tendrá a su vez dentro el componente <Menu /> y el resto de componentes que tenga la página /home. Lo mismo pasa si ponemos a mano en el navegador la URL <http://localhost:5173/reports>, que el enrutador nos montará el componente <Reports> que a su vez tiene dentro el componente <Menu> y el resto de componente que tenga esa página. Es decir, entramos directamente en cualquiera de las dos páginas sin poner usuario ni contraseña.

Para evitar que pase eso se usa el hook useEffect(). (<https://react.dev/reference/react/useEffect>)

Como el componente común a ambas páginas (la de /home y la de /reports) es el que creamos en el punto 3, es ahí donde tendremos que colocar el hook useEffect().

Este hook permite sincronizar un componente cuando cambie algo externo. Ese algo externo se representa con las dependencias. Es decir, si cambian las dependencias, yo haré una acción en mi componente. Esas dependencias deben estar dentro de la lógica del `useEffect()`. En nuestro caso, si no estamos autenticados, queremos que se navegue a /. Por lo tanto las dependencias son estar o no autenticados y navega

Esta es la estructura de uso del hook `useEffect`:

```
useEffect(() => { //Lo que queremos hacer }, [dependencias])
```

Para saber si estamos o no autenticados, usamos los datos que obtuvimos del store con el `useSelector()`. Esos datos están almacenados en el objeto `userData` (o como lo hayan llamado ustedes en su código). Recordemos que ese objeto tiene los siguientes elementos, que nos dicen el nombre de usuario, el rol del usuario y si está o no autenticado:

```
userData.userName userData.userRol userData.isAuthenticated
```

Con el hook `useEffect()` evitamos que se monte la página `/home` o la página `/reports`, puesto que reaccionará cuando se cambie el `userData.isAuthenticated`, comprobará si es `false` y si lo es, navegaremos a `http://localhost:5173/` (es decir, la ruta padre `/`).

```
//Para usar el useEffect debemos importarlo
import { useEffect } from 'react'
```

```
//Trozo de código donde vamos a usar el useEffect(): siempre los hooks van al principio del
componente
```

```
//Antes de esto tendremos que coger del store los datos
const isLoggedIn = userData.isAuthenticated
```

```
useEffect(() => {

  if (!isLoggedIn) {
    navigate('/')
  }
}, [isLoggedIn, navigate])
```

Comprueba ahora que al poner `http://localhost:5173/home` o `http://localhost:5173/reports` directamente en la URL no te deja entrar puesto que no estás logueado.

Primero importo

```
import { useEffect } from 'react'
```

```
//Almacenamos en la variable userData lo que obtenemos del store usando el hook useSelector
const userData = useSelector((state: RootState) => state.authenticator)
//Comprobamos por la consola qué obtenemos del store
console.log(userData)

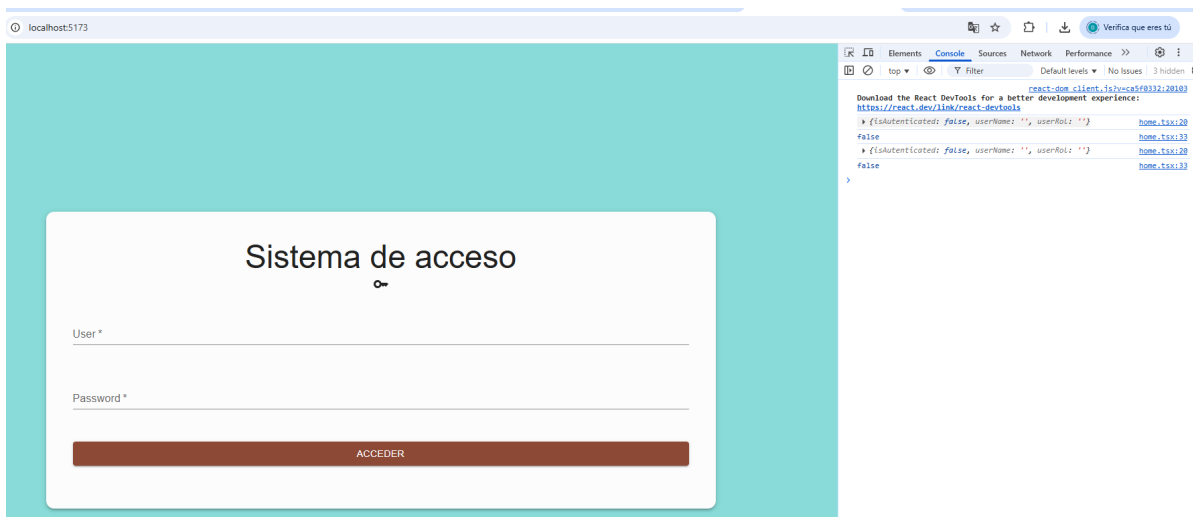
function HandleAction(){
  dispatch(authActions.logout())
  //Uso el mismo dispatch para salir que para entrar solo que en vez de login, pongo logout y como es un metodo voy que solo it
  //no me pide ningun parametro como nombre o rol
  navigate("/")
  //navigate() es para ir al root A.K.A el login de inicio
  //todo esto es simplemente un onclick del boton de mas abajo en el que solamente pone salir
}
//Antes de esto tendremos que coger del store los datos
const isLoggedIn = userData.isAuthenticated
//Tiene que ser magia, puse esto y empezo a funcionar, así se va a quedar
console.log(isLoggedIn)

useEffect(() => {
  if (!isLoggedIn) {
    navigate('/')
  }
}, [isLoggedIn, navigate])
```

En rojo se ve como declaro el userData, luego debajo del handleAction esta la variable de login que usare para saber si el user esta logeado y sin ningun tipo de log embrujado el useEffect funciona de tal manera que si el usuario no esta logeado mediante la variable isLoggedIn lo manda al login



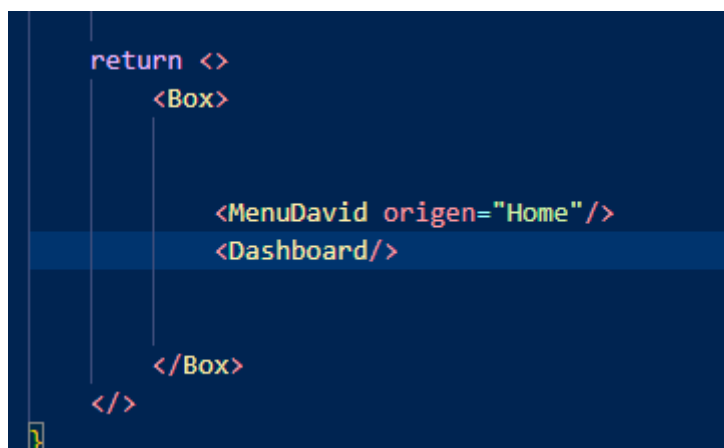
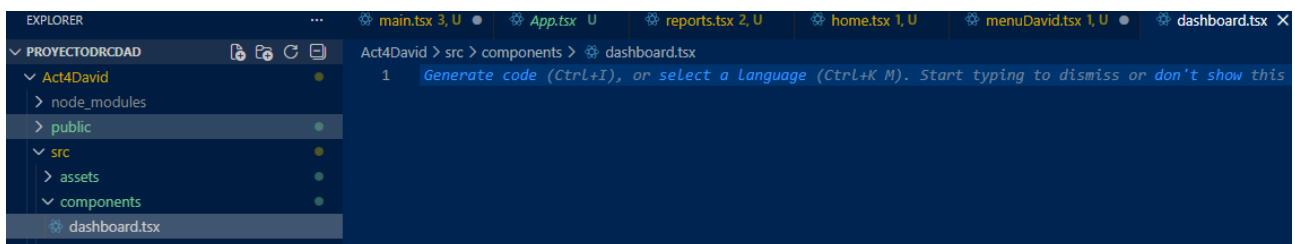
Aquí intento entrar al home pero no estoy autenticado por lo que me manda para atrás, lo demuestro con este log mágico que hace que las cosas funcionen como dice en el código



5. Creación del componente que usarás en Home.tsx

Crea un componente llamado Dashboard en el fichero frontend/src/components/Dahsboard.tsx. Este componente tendrá el formulario y la tabla que se usarán en la página /home.

Es decir, ahora el fichero Home.tsx renderizará los componentes menu y dashboard en su return:



6. Creación del formulario en Dashboard.tsx

Ya has creado varios formularios a lo largo del curso, así que esto no debería ser complicado. Ten en cuenta que el formulario debe ser responsive

Recuerda que todos los elementos del formulario: nombre, tipo, marca y precio los debes poner en un useState. Como estamos trabajando con Typescript deben indicar los tipos de los diferentes elementos, para ello hay que crear una interface:

```
import React, { useState } from "react";
import { Box, TextField, Button } from "@mui/material";

type FormData = {
  id?: number;
  nombre: string;
  marca: string;
  tipo: string;
  precio: number;
};

export function dashboard() {
  const [formData, setFormData] = useState<FormData>({
    nombre: "",
    marca: "",
    tipo: "",
    precio: 0
  });
}
```

Primero hago los imports y declaro las variables que introduciré, son 2 arrays

```
return (
  <>
    <Box
      sx={{
        width: "600px",
        display: "flex",
        gap: 2,
        marginTop: 3,
      }}
    >
      <TextField
        label="ID (opcional)"
        type="number"
        name="id"
        value={formData.id ?? ""}
        onChange={handleChange}
        fullWidth
      />
    </Box>
  </>
)
```

Declaro el box donde estarán los 5 campos con un poco de formato para que se pueda leer y muestro uno de los 5 textfield, son todos iguales cambiando los datos correspondientes a cada campo

```

    </Box >
    <Box sx={{padding:"15px"}}>
      <Button variant="contained" sx={{width:"400px"}}>
        Guardar
      </Button>
    </Box>
  </>
);

```

Por ultimo el boton para guardar

Home de david, Hola soy david y tengo el rol de administrador

SALIR

ID (opcional) Nombre Marca Tipo Precio

0

GUARDAR

Asi me queda el formulario

El botón de + INSERTAR DATOS de nuestro formulario debe insertar datos en la tabla coleccion de nuestra base de datos.

Para ello debemos responder al evento onSubmit realizando una llamada al endpoint correspondiente.

Incisio para la creacion de la base de datos y todo lo relacionado al backend

Estás creando una aplicación web con React para gestionar una colección de objetos, ya sea de música, de ropa, de videojuegos, juegos de mesa, cartas de pokemon, etc. Tú eliges de qué vas a hacer la colección.

A través del login cuya interfaz ya tienes creada, van a poder acceder dos tipos de usuario:

. Usuario con el rol de admin: este usuario podrá acceder a la colección y realizar las operaciones de insertar, borrar y actualizar en la base de datos a través de la interfaz que crearás en la página home

. Usuario con el rol user: este usuario podrá acceder a la colección y realizar sólo la operación de insertar en la base de datos a través de la interfaz.

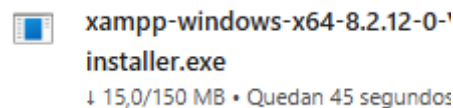
Base de datos del proyecto

En el campus, en la parte de Base de datos y Ficheros para el backend (en la parte de Recursos de UT2) tienes el fichero bdgestion.sql que es una base de datos MySQL con dos tablas:

- . Tabla coleccion: tabla con un id (clave principal), nombre, marca, tipo y precio. La tabla está vacía
- . Tabla usuarios: tabla con los usuarios que pueden acceder a la aplicación web

La tabla de usuarios ya tiene dos usuarios predefinidos: Patricia con el rol de admin y usuario con el rol de user

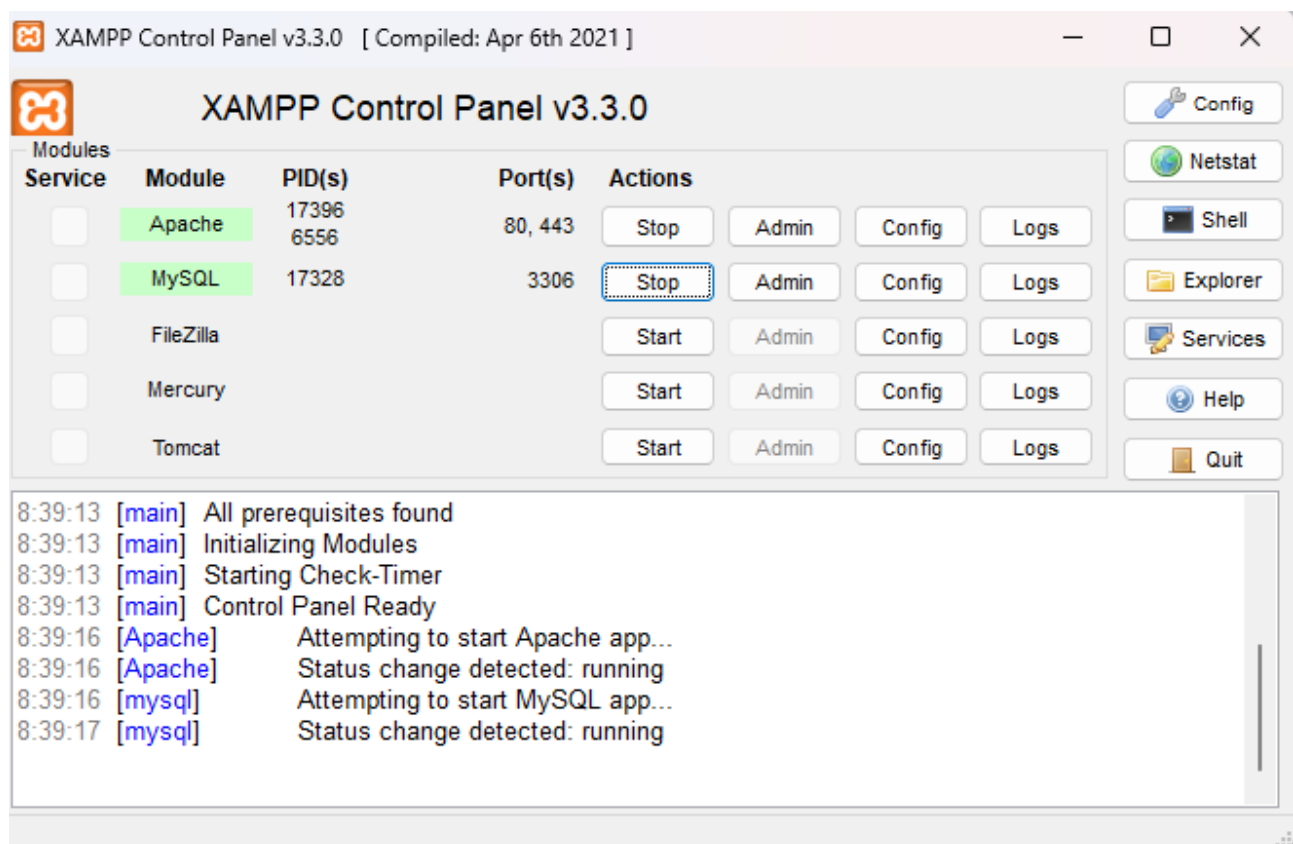
Para poder usar la base de datos tenemos que instalar XAMPP.



<https://www.apachefriends.org/es/index.html>

El XAMPP tiene un servidor Apache y un módulo MySQL. Esos dos son los que vamos a usar. Al pica en Start iniciamos ambos servicios:

Al picar en Admin en MySQL se nos abre el administrador de la base de datos
<http://localhost/phpmyadmin/>



Reciente Favoritas

Nueva

- information_schema
- mysql
- performance_schema
- phpmyadmin
- test

Bases de datos

Crear base de datos

bdGestion

utf8mb4_general_ci

Crear

☐ Seleccionar todo

Eliminar

Base de datos

Cotejamiento

Acción

localhost/phpmyadmin/index.php?route=/database/import&db=bdgestion

phpMyAdmin

Reciente Favoritas

Nueva

- bdgestion
- information_schema
- mysql
- performance_schema
- phpmyadmin
- test

Servidor: 127.0.0.1 » Base de datos: bdgestion

Estructura SQL Buscar Generar una consulta Exportar Importar Operaciones Privilegios

Importando en la base de datos "bdgestion"

Archivo a importar:

El archivo puede ser comprimido (gzip, bzip2) o descomprimido.
Un archivo comprimido tiene que terminar en `[formato].[compresión]`. Por ejemplo: `.sql.zip`

Buscar en su ordenador: (Máximo: 40MB)

Seleccionar archivo bdgestion.sql

También puede arrastrar un archivo en cualquier página.

Conjunto de caracteres del archivo:

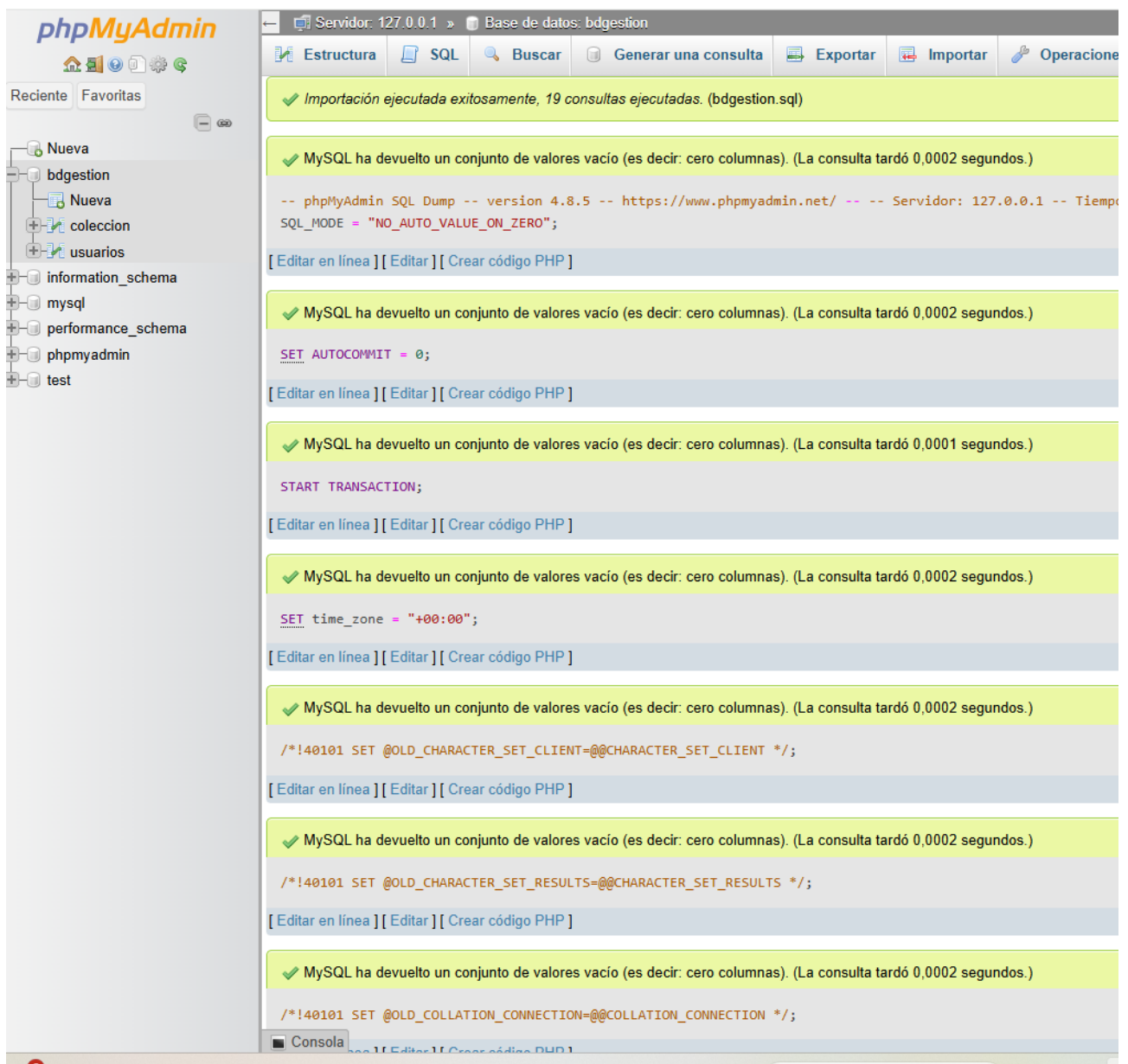
utf-8

Importación parcial:

☒ Permitir la interrupción de una importación en caso que el script detecte que se ha acercado al límite de tiempo PHP.
Esta puede ser una buena forma para importar archivos grandes, sin embargo, puede romper las transacciones.

Omitir esta cantidad de consultas (en SQL) desde la primera:

0



Creación del backend en el proyecto: endpoints

Nuestro backend será una API a través de la cual el frontend obtendrá, eliminará e insertará datos en la base de datos

En la carpeta de ProyectoMisiniciales crea la carpeta del backend:

```
mkdir backend  
cd backend
```

Dentro del backend usa el comando npm init para crear el fichero package.json puesto que lo necesita el node.js

```
npm init
```

Te pide los datos del fichero package.json, puedes dar a enter a todo o poner algunos como el author y la description.

```
PS C:\Users\alumnadoM\Desktop\proyectoDRCDAD\Act4David\backend> npm init
This utility will walk you through creating a package.json file.
It only covers the most common items, and tries to guess sensible defaults.

See `npm help init` for definitive documentation on these fields
and exactly what they do.

Use `npm install <pkg>` afterwards to install a package and
save it as a dependency in the package.json file.

Press ^C at any time to quit.
package name: (backend) backend
version: (1.0.0)
description: back del proyecto 4
entry point: (index.js)
test command:
git repository:
keywords:
author: David
license: (ISC)
About to write to C:\Users\alumnadoM\Desktop\proyectoDRCDAD\Act4David\backend\package.json:

{
  "name": "backend",
  "version": "1.0.0",
  "description": "back del proyecto 4",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "author": "David",
  "license": "ISC"
}

Is this OK? (yes) yes

PS C:\Users\alumnadoM\Desktop\proyectoDRCDAD\Act4David\backend> █
```

Instalación de las librerías necesarias para programar los endpoints del backend (API)

El frontend de nuestra aplicación se va a comunicar con la base de datos a través de endpoints. Un endpoint (punto final de aplicación) es una URL (una dirección) de la API que se encarga de contestar una petición del frontend (cliente) enviada a través de métodos de solicitud HTTP. La contestación de los endpoint se trata en formato JSON.

Somos nosotros los programadores los que definimos los endpoints. En principio, en nuestra API crearemos un endpoint llamado /login para obtener datos referentes al usuario en la tabla usuarios de nuestra base de datos. Más adelante, necesitaremos otros endpoints para seleccionar, insertar, actualizar y borrar datos en la tabla coleccion. En los endpoints es donde haremos las consultas SQL. Es por eso que estas funciones serán serás asíncronas.

Para trabajar con nuestro backend necesitamos instalar las librerías express y cors. Express es un framework web para node y cors nos permite que la aplicación tenga acceso a la API que estamos creando (desde otro servidor o, como es nuestro caso, desde localhost). Cors nos permite tener tanto la API como la aplicación en localhost

Así pues instalamos las librerías en nuestra carpeta de backend:

```
cd backend  
npm install express  
npm install cors
```

Vemos que están instaladas en el fichero package.json que está dentro de la carpeta backend.

```
PS C:\Users\alumnadoM\Desktop\proyectoDRCDAD\Act4David\backend> npm install express  
  
added 66 packages, and audited 67 packages in 4s  
  
18 packages are looking for funding  
  run `npm fund` for details  
  
found 0 vulnerabilities  
PS C:\Users\alumnadoM\Desktop\proyectoDRCDAD\Act4David\backend> npm install cors  
  
added 2 packages, and audited 69 packages in 1s  
  
18 packages are looking for funding  
  run `npm fund` for details  
  
found 0 vulnerabilities  
PS C:\Users\alumnadoM\Desktop\proyectoDRCDAD\Act4David\backend> █
```

Instalación de la librería para poder trabajar con base de datos MySQL

En la carpeta del backend instalamos la librería para trabajar con base de datos MySQL:

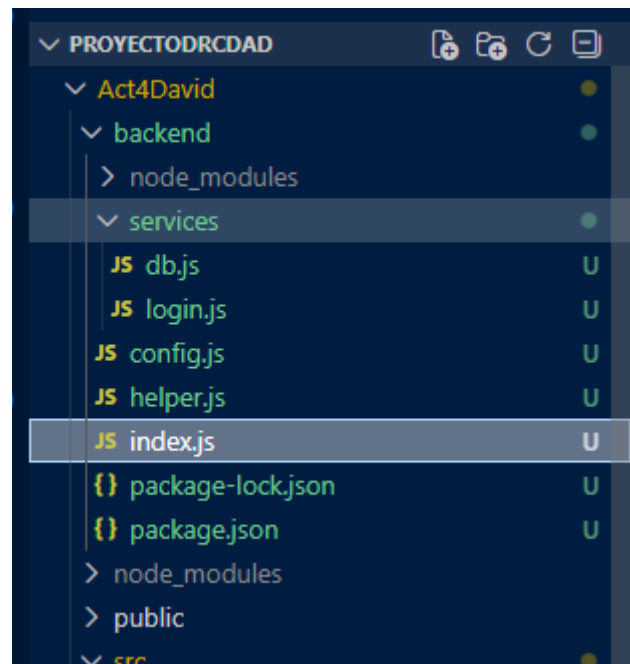
```
cd backend  
npm install mysql2
```

```
found 0 vulnerabilities  
PS C:\Users\alumnadoM\Desktop\proyectoDRCDAD\Act4David\backend> npm install mysql2  
  
added 12 packages, and audited 81 packages in 2s  
  
19 packages are looking for funding  
  run `npm fund` for details  
  
found 0 vulnerabilities  
PS C:\Users\alumnadoM\Desktop\proyectoDRCDAD\Act4David\backend> █
```

Creación del endpoint del login y otro de ejemplo

Para esta parte usas el resto de ficheros que te descargaste del Campus en Base de datos y Ficheros para el backend (en la parte de Recursos de UT2). Tienen los ficheros: index.js, helper.js, config.js, db.js y login.js

Deben descargarlos y copiarlos en la carpeta backend del proyecto siguiendo la siguiente estructura:



En la carpeta backend copias los ficheros: config.js, helper.js e index.js. Luego crea la carpeta services dentro de backend. Ahí dentro copias los ficheros db.js y login.js.

-Fichero index.js: es donde importamos las librerías que vamos a usar y los ficheros que necesitamos para los endpoints. Definimos el puerto (3030) de nuestra API y también los diferentes endpoints.

-Fichero config.js: aquí definimos las credenciales de nuestra base de datos.

-Fichero helper.js: para manejar casos en los que la base de datos no devuelve nada.

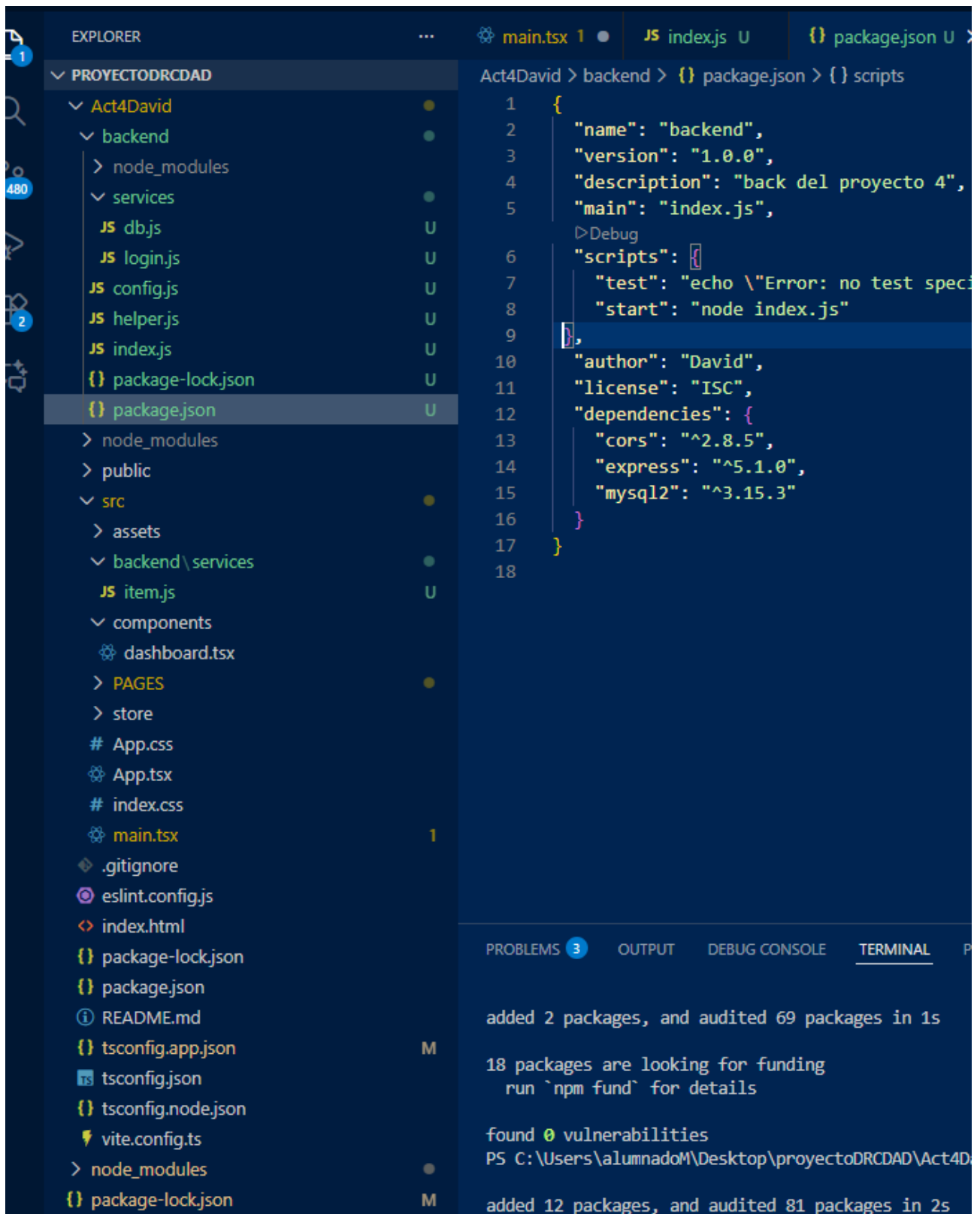
-Fichero services/login.js: aquí es donde creamos la consulta para comprobar que los datos proporcionados por el usuario en el frontend se encuentran en la base de datos.

-Fichero services/db.js: aquí creamos la conexión con la base de datos y nos conectamos a la misma realizando la consulta que nos pasan.

Para ejecutar el fichero index.js tenemos dos opciones:

- .Escribir `node index.js` en la consola

- .Añadir, en el fichero `package.json`, en la sección de scripts, la siguiente línea correspondiente a "start":



Un vez hecho esto vamos a probar nuestra API. Iniciamos la API con el siguiente comando, dentro de la carpeta backend:

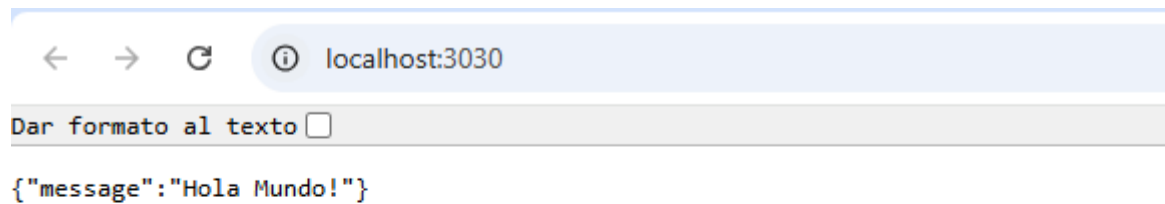
npm start

```
found 0 vulnerabilities
PS C:\Users\alumnadoM\Desktop\proyectoDRCAD\Act4David\backend> npm start

> backend@1.0.0 start
> node index.js

API escuchando en el puerto 3030
```

Abrimos un navegador en el puerto 3030 que es donde se está ejecutando el backend (la API):
<http://localhost:3030/>



localhost:3030

Dar formato al texto ☐

```
{"message": "Hola Mundo!"}
```

Aquí estamos viendo lo que está programado en el endpoint / que está en el fichero index.js. Probamos a cambiar el mensaje en el fichero index.js y vemos cómo cambia. Veremos que para aplicar el cambio tenemos que guardar el fichero, parar la ejecución de la API y volver a ejecutarla.

Para no tener que parar la API cada vez que hacemos cambios vamos a instalar el paquete nodemon en la carpeta del backend:

```
cd backend
npm install nodemon
```

```
API escuchando en el puerto 3030
PS C:\Users\alumnadoM\Desktop\proyectoDRCAD\Act4David\backend> npm install nodemon

added 27 packages, and audited 108 packages in 2s

23 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

Ahora cambio en el package.json el script de start:

```
"scripts": {
  "test": "echo \"Error: no test specified\" && exit 1",
  "start": "nodemon index.js"
},
"author": "David"
```

Cuando ejecuto ahora el proyecto en la consola con npm start, vemos que nos permite hacer cambios en la API y no se para:

```
found 0 vulnerabilities
PS C:\Users\alumnadoM\Desktop\proyectoDRCAD\Act4David\backend> npm start

> backend@1.0.0 start
> node index.js

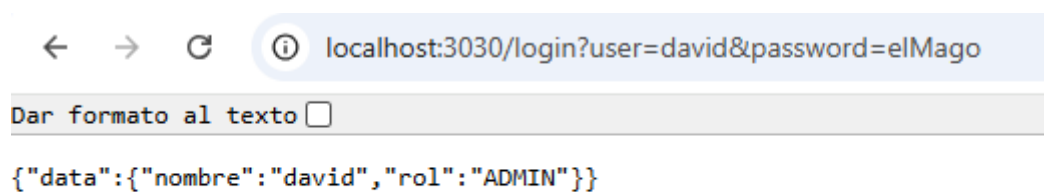
API escuchando en el puerto 3030
```

Supongamos que funciona

Acceder a los endpoints desde el navegador

Ya hemos visto en el ejemplo anterior como cuando ponemos la URL: <http://localhost:3030/> estamos accediendo al endpoint /, el cual hemos programado en el fichero index.js

De la misma forma podemos acceder a cualquier endpoint que tengamos programado, como por ejemplo el endpoint de /login. En este endpoint vemos que tenemos que pasarle dos parámetros, el user y el password. Así que en la URL tenemos que poner lo siguiente:

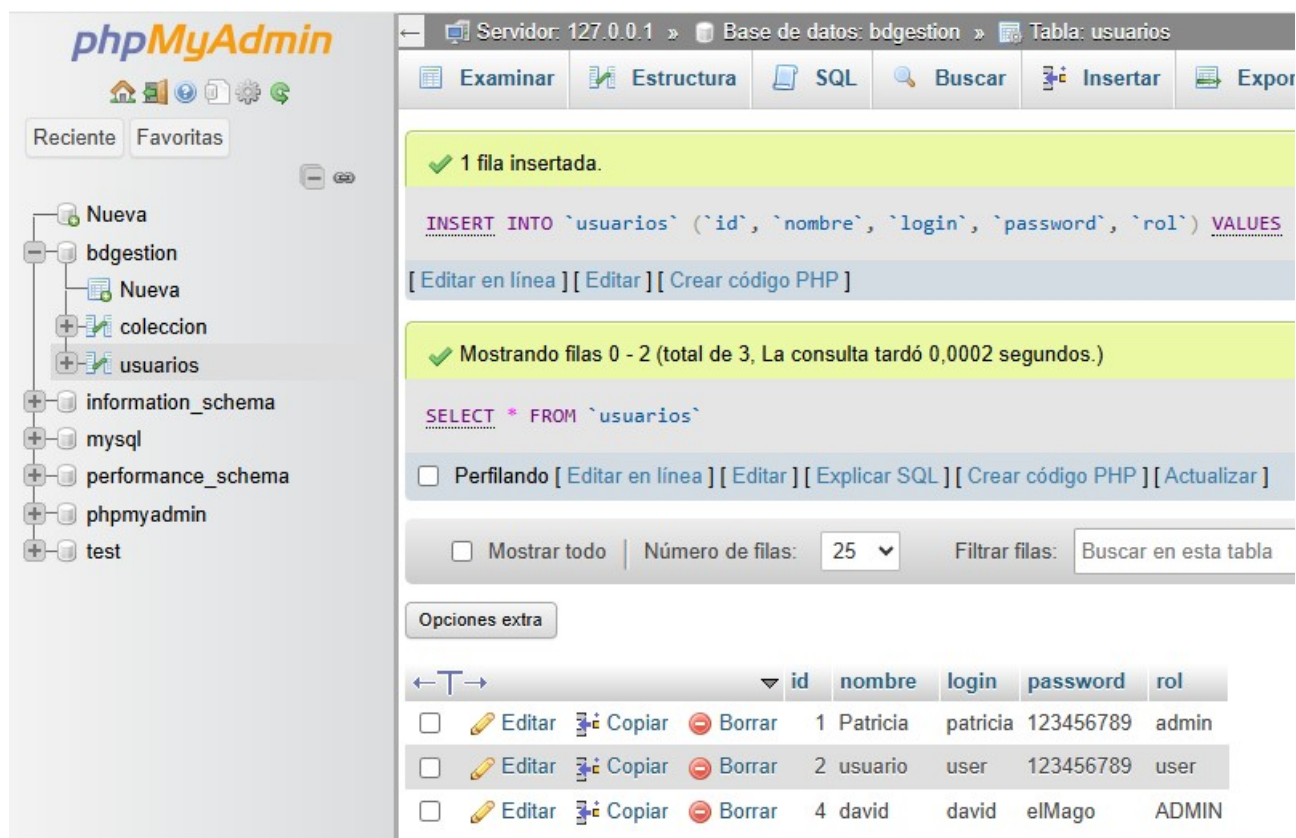


← → ↻ ⓘ localhost:3030/login?user=david&password=elMago

Dar formato al texto ☐

```
{"data":{"nombre":"david","rol":"ADMIN"}}
```

Aquí entra con un login creado por mí, en la tarea usa uno creado ya, lo que hice fue añadir un usuario en la base de datos creada anteriormente y use esas credenciales



phpMyAdmin

Reciente Favoritas

Nueva

- bdgestion
 - Nueva
 - coleccion
 - usuarios
- information_schema
- mysql
- performance_schema
- phpmyadmin
- test

Servidor: 127.0.0.1 » Base de datos: bdgestion » Tabla: usuarios

Examinar Estructura SQL Buscar Insertar Exportar

✓ 1 fila insertada.

```
INSERT INTO `usuarios` (`id`, `nombre`, `login`, `password`, `rol`) VALUES
```

[Editar en línea] [Editar] [Crear código PHP]

✓ Mostrando filas 0 - 2 (total de 3, La consulta tardó 0,0002 segundos.)

```
SELECT * FROM `usuarios`
```

☐ Perfilando [Editar en línea] [Editar] [Explicar SQL] [Crear código PHP] [Actualizar]

☐ Mostrar todo | Número de filas: 25 | Filtrar filas: Buscar en esta tabla

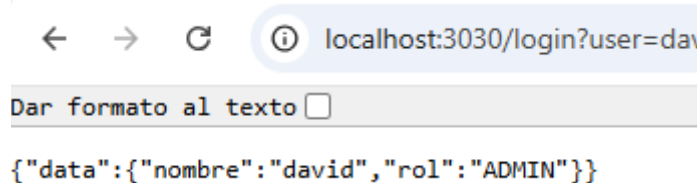
Opciones extra

					id	nombre	login	password	rol
☐	✎ Editar	📋 Copiar	🗑 Borrar		1	Patricia	patricia	123456789	admin
☐	✎ Editar	📋 Copiar	🗑 Borrar		2	usuario	user	123456789	user
☐	✎ Editar	📋 Copiar	🗑 Borrar		4	david	david	elMago	ADMIN

Aquí la bd

Fíjate que estamos accediendo al endpoint login (/login) y le estamos pasando el parámetro user con el valor david y el parámetro password con el valor elMago. Esto llama a la función que realiza la consulta en la base de datos y nos devuelve los datos de ese usuario que están almacenados en la base de datos.

Vemos que nos devuelve en formato JSON el data que tiene dentro el nombre y el rol.

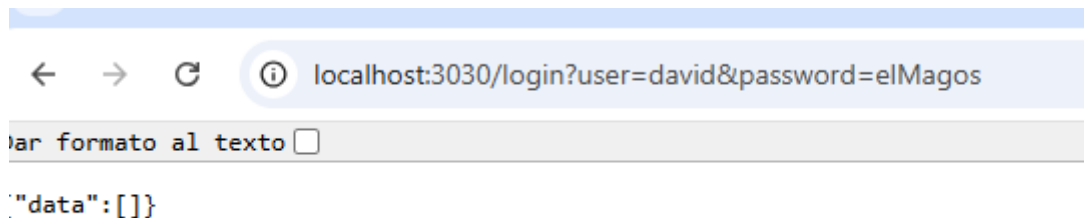


localhost:3030/login?user=david

Dar formato al texto ☐

```
{"data":{"nombre":"david","rol":"ADMIN"}}
```

Si me equivoco adrede la base de datos me devuelve datos vacíos:



¿Cómo llamar a los endpoints desde el frontend?

Para llamar a los endpoints desde el frontend se usa un fetch, que nos sirve para acceder y manipular llamadas HTTP. El fetch lo tendremos que colocar en la parte del código donde vamos a comprobar si el usuario y contraseña son los correctos (una vez que piquemos en el botón ACCEDER).

Ejemplo de llamada al endpoint /login desde el componente del frontend y de cómo tenemos que esperar la respuesta (al ser una llamada asíncrona):

```
async function isVerifiedUser () {  
  fetch(`http://localhost:3030/login?user=${data.user}&password=${data.passwd}`)  
    .then(response => response.json())  
    .then (response => {  
      console.log('Lo que nos llega de la base de datos: ')  
      console.log(response.data)  
      if (response.data.length !== 0){  
        //Si hay datos es que el usuario y contraseña son los correctos. Hago el dispatch y el  
        //navigate  
      } else{  
        //Si no, realizo la lógica para alertar al usuario con usuario/contraseña son incorrectas  
      }  
    })  
}
```

El fetch se suele poner dentro de una función asíncrona puesto que esperamos por la respuesta de la base de datos. Con el fetch llamamos al endpoint /login pasándole el login de usuario y la contraseña. En este caso, debemos usar las variables donde se almacenan el login (el nombre que usa el usuario para ingresar en la aplicación) y la contraseña.

Después de llamar espero la respuesta (response) que la paso a json. Si hay respuesta, la mando a la consola para ver lo que trae la base de datos (esto lo tendrías que eliminar cuando la aplicación está en producción). Si la longitud de los datos que trae de la base de datos es distinta de cero es que encontró el usuario y la contraseña, por lo tanto son correctos.

7. Creación del fichero backend/services/items.js

Ahora vamos a saltar al backend de la aplicación para crear las consultas que manejarán la comunicación con la tabla coleccion de la base de datos bdgestion.sql. Contenido incompleto del fichero que irán rellenando:

```
const db = require('../../../backend/services/db.js')
const helper = require('../../../backend/helper.js')
const config = require('../../../backend/config.js')

// INSERT
async function insertData (req, res) {

  // Los datos llegan por req.query
  const data = req.query

  const result = await db.query(
    [data.nombre, data.marca, data.tipo, data.precio]
    `
    INSERT INTO coleccion (nombre,marca,tipo, precio)
    VALUES (?, ?, ?, ?)
    `,
  )

  // Devuelve cuántas filas fueron insertadas
  return result.affectedRows
}

// SELECT
async function getData (req, res) {

  const rows = await db.query(`
    SELECT * FROM coleccion
  `)

  const data = helper.emptyOrRows(rows)

  return {
    data
  }
}

// DELETE
async function deleteData (req, res) {

  // Recogemos el id por req.query
  const data = req.query

  const result = await db.query(
    [data.id]
    `
    DELETE FROM coleccion
    WHERE id = ?
    `,
  )

  return result.affectedRows
}

// Exportación de funciones
module.exports = {
  getData,
  insertData,
  deleteData
}
```


8. Añadir funcionalidad al botón Insertar del formulario.

Cuando se pique en el botón Insertar, hay que insertar los datos en la base de datos y avisar al usuario que se han insertado correctamente.

a) Programar la consulta INSERT en la función insertData del fichero backend/services/items.js

```
// INSERT
async function insertData (req, res) {

  // Los datos llegan por req.query
  const data = req.query

  const result = await db.query(
    [data.nombre, data.marca, data.tipo, data.precio]
    `
    INSERT INTO coleccion (nombre,marca,tipo, precio)
    VALUES (?, ?, ?, ?)
    `
  )

  // Devuelve cuántas filas fueron insertadas
  return result.affectedRows
}
```

b) Añadir en el fichero backend/index.js un endpoint para añadir un registro en la base de datos. Yo al endpoint le puse el nombre /addItem pero lo pueden nombrar como quieran. Fíjate en el endpoint /login, es muy parecido el código. El código incompleto sería así:

```
app.get('/addItem', async function(req, res, next) {
  try {
    res.json(await insercion.insertData(req.query.nombre, req.query.marca,
    req.query.tipo ,req.query.precio))
  } catch (err) {
    console.error(`Error while inserting items `, err.message);
    next(err);
  }
})
```

c) En el frontend llamar al backend con la función `fetch()` cuando se pique el botón de Insertar. En la función que maneje el botón Insertar, debemos hacer un fetch al endpoint `/addItem`. Aquí fijarse en el fetch que hicieron en el login, es muy parecido. Si la respuesta del fetch es `> 0`, lanzamos un alert indicando que hemos insertado los datos correctamente (`alert('Datos guardados con éxito')`).

```
/* | FUNCION QUE INSERTA EN LA BD */
const handleInsert = async () => {
  const { nombre, marca, tipo, precio } = formData;

  const response = await fetch(
    `/addItem?nombre=${nombre}&marca=${marca}&tipo=${tipo}&precio=${precio}`
  );

  const data = await response.json();

  if (data > 0) {
    alert("Datos guardados con éxito");
  } else {
    alert("No se pudo insertar en la base de datos");
  }
};
```

9. Creación del resto de los endpoints: `/getItems` y `/deleteItem`

a) Programar la consulta `SELECT` en la función `getData` del fichero `backend/services/items.js`

```
// SELECT
async function getData (req, res) {

  const rows = await db.query(`
    SELECT * FROM coleccion
  `)

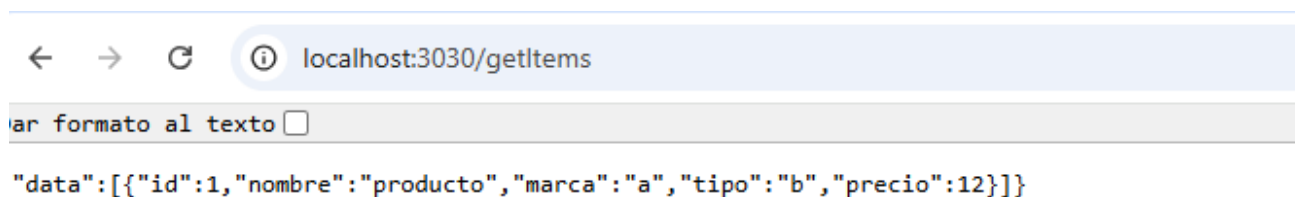
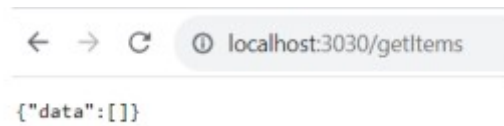
  const data = helper.emptyOrRows(rows)

  return {
    data
  }
}
```

b) Añadir en el fichero backend/index.js el endpoint para seleccionar los datos de la tabla colección. Yo al endpoint le puse el nombre /getItems pero lo pueden nombrar como quieran. El código incompleto sería así:

Para probar que funciona vamos al endpoint /getItems a través del navegador (recuerden que los endpoints están en el puerto 3030): `http://localhost:3030/getItems` Veremos cómo son los datos que nos devuelve la base de datos. Si en la tabla colección la base de datos aún está vacía nos devuelve un objeto vacío:

Para probar que funciona vamos al endpoint /getItems a través del navegador (recuerden que los endpoints están en el puerto 3030): `http://localhost:3030/getItems` Veremos cómo son los datos que nos devuelve la base de datos. Si en la tabla colección la base de datos aún está vacía nos devuelve un objeto vacío:



Me devolvía un objeto vacío, pero añadí una inserción en la db y ahora se muestra

Si insertamos algún registro en la tabla colección veremos que nos devuelve un objeto data que tiene dentro un array de objetos. Cada objeto es un registro de la tabla colección:

9.2 Borrar un registro de la base de datos (/deleteItem)

a) Programar la consulta DELETE en la función deleteData del fichero backend/services/items.js

```
// DELETE
async function deleteData(req) {
  const data = req.query;

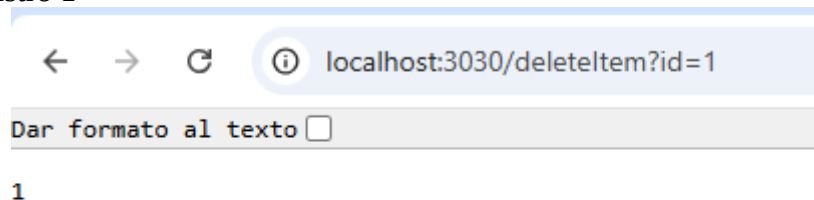
  const result = await db.query(
    `
    DELETE FROM coleccion
    WHERE id = ?
    `,
    [data.id]
  );

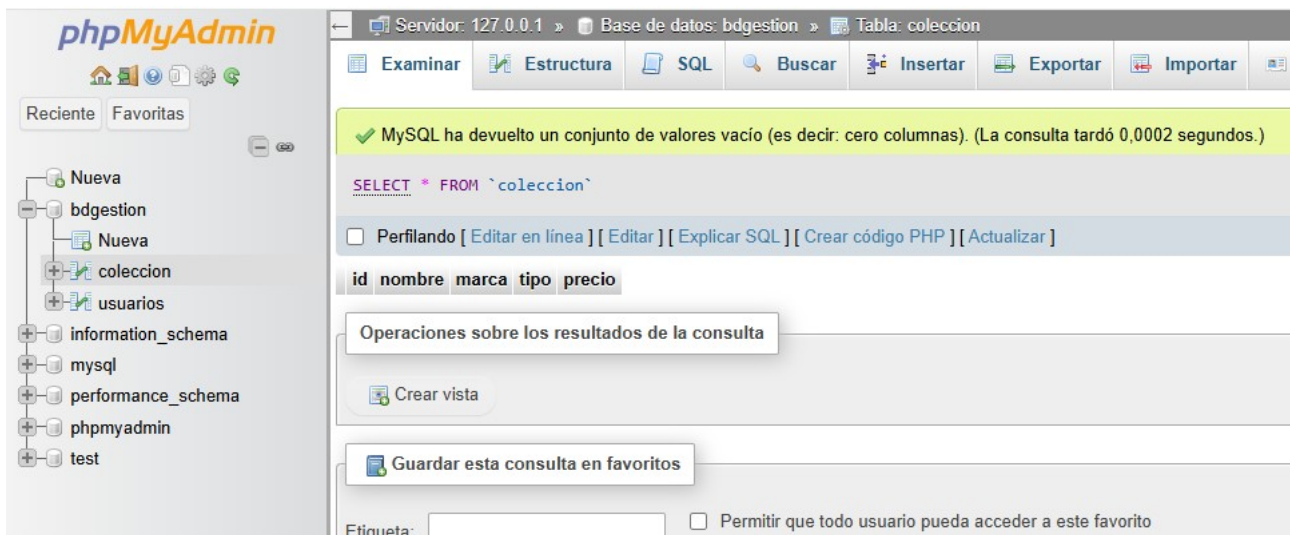
  return result.affectedRows;
}
```

b) Añadir en el fichero backend/index.js el endpoint para seleccionar los datos de la tabla coleccion. Yo al endpoint le puse el nombre /deleteItem pero lo pueden nombrar como quieran. El código incompleto sería así:

```
app.get('/deleteItem', async function(req, res, next) {
  try {
    res.json(await deleteData(req))
  } catch (err) {
    console.error(`Error while deleting items `, err.message);
    next(err);
  }
})
```

Aquí borro el registro 1





Aquí se borra el registro en la db

10. Creación de la tabla en el fichero Dashboard.tsx: lógica para mostrar los datos en la tabla y lógica para borrar un registro de la tabla

10.1 Lógica para mostrar los datos en la tabla

Con la tabla mostramos los datos que están almacenados en la tabla coleccion de nuestra base de datos. Para mostrar los datos tenemos que hacer un SELECT de la tabla colección, almacenar dichos datos en una variable (que será un array) y luego mostrar los datos de la variable en la tabla.

Así que lo primero que tendremos que hacer es crear una variable en el fichero frontend/src/components/Dashboard.tsx en la que almacenaremos los datos obtenidos del SELECT. Como esa variable va a actualizarse cada vez que hagamos el select, la ponemos como un useState. Su valor inicial será un array vacío. Cuando hagamos el select, su valor se actualizará con los valores obtenidos de la base de datos.

```
const [tableData, setTableData] = useState([])
```

Para pensar ... ¿En qué parte del frontend hacemos fetch() al endpoint que hace el SELECT? ¿Cómo lo hacemos?

```
</Box>
  <Box sx={{padding:"15px"}}>
    <Button variant="contained" sx={{width:"400px"}} onClick={handleSelect}>
      Seleccionar
    </Button>
  </Box>
</>
```

Yo lo añadí en el botón

Una vez tenemos claro en qué parte hacemos el `fetch()` al endpoint que realiza el `SELECT` y tenemos ya almacenados los datos de la tabla colección en la variable `tableData` ... vamos a crear la tabla.

10.2 Implementación de la tabla: componente Table

Para la tabla usamos el componente `Table` de la librería `@MUI`:

<https://mui.com/material-ui/reacttable/>

Una tabla tiene la siguiente estructura básica:

- Contenedor de tabla (`TableContainer`)
- La tabla en sí (`Table`)
- La cabecera de la tabla (`TableHead`) con un componente fila (`TableRow`) y luego cada una de las celdas (`TableCell`) de la fila de la cabecera
- El cuerpo de la tabla (`TableBody`) que tendrá varios componentes fila con varias celdas cada uno de ellos

En nuestro caso vamos a rellenar el cuerpo de la tabla con los datos de la base de datos, que vienen en un array de objetos. Para recorrer un array en javascript se usa el método `map` como ya saben. Así pues, vemos a continuación el esquema de una tabla y la parte donde se haría el `.map` del array:

Home de david, Hola soy david y tengo el rol de administrador

SALIR

Nombre: Coche, Marca: Toyota, Tipo: movil, Precio: 12

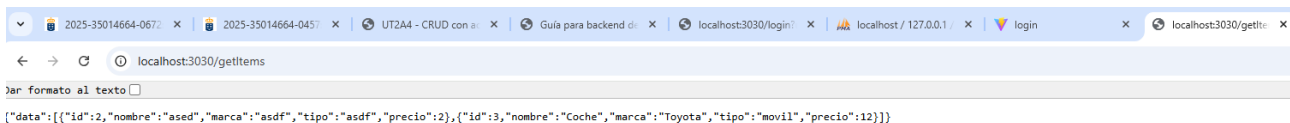
GUARDAR

Aquí ya inserto los datos

SELECCIONAR			
Nombre	Marca	Tipo	Precio
ased	asdf	asdf	2
Coche	Toyota	movil	12



Y aquí tengo los datos reflejados en la bd



Y aquí la orden del index

Lo que queremos hacer ahora es mostrar con el componente esos datos obtenidos del endpoint /getItems y almacenados en el array tableData. Nos centramos ahora en lo que se pondría dentro del Body de la tabla, que es un .map con el que recorremos el array tableData (siempre y cuando haya datos, si no los hay no se entra en el map). La variable row toma el valor de los elementos del array a medida que se va recorriendo. Así que row.id será el valor del id, row.nombre será el valor del nombre, ... Fíjense que row es de tipo itemtype, que fue el que definimos al principio del código de Home.tsx. Fíjense también que para el key usamos el id que proviene de la base de datos.

```
<TableContainer aria-label="Tabla david" component={Paper}>
  <Table sx={{ minWidth: 650 }} aria-label="simple table">
    <TableHead>
      <TableRow>
        <TableCell>Nombre</TableCell>
        <TableCell align="right">Marca</TableCell>
        <TableCell align="right">Tipo</TableCell>
        <TableCell align="right">Precio</TableCell>
      </TableRow>
    </TableHead>
    <TableBody>
      {tableData.map((row) => (
        <TableRow
          key={row.id}
          sx={{ '&:last-child td, &:last-child th': { border: 0 } }}
        >
          <TableCell component="th" scope="row">
            {row.nombre}
          </TableCell>
          <TableCell align="right">{row.marca}</TableCell>
        </TableRow>
      )
    )}
  </Table>
</TableContainer>
```

```

<TableCell align="right">{row.tipo}</TableCell>
<TableCell align="right">{row.precio}</TableCell>
</TableRow>
)}}
</TableBody>
</Table>
</TableContainer>

```

Estas es la generacion de mi tabla

10.3 Lógica para borrar un registro de la tabla

En el código del TableBody hemos insertado un componente con un componente icono (). Al picar en dicho botón icono, tendremos que implementar una función a la que pasamos el id del registro que queremos borrar. Dicha función hará el fetch al endpoint /deleteItem. Ahora les toca a ustedes ...

Ya hice la funcion para seleccionar el id en el formulario y ahi decir según lo que veo en la tabla que registro borrar

```

3   </TableContainer>
4
5
6   <Box sx={{ padding: "15px" }}>
7     <Button
8       variant="contained"
9       sx={{ width: "400px" }}
10      onClick={handleDelete}
11      startIcon={<DeleteForeverIcon />}
12    >
13  </Button>

```

Este es el boton que uso para llamar a la funcion


```

const handleDelete = async () => {
  const id = formData.id;
  try {
    const response = await fetch(
      `http://localhost:3030/deleteItem?id=${id}`
    );
    const result = await response.json();
    setTableData(result.data);
  } catch (error) {
    console.error('Error fetching data:', error);
  }
};

```

Aqui llamo al endpoint

```

app.get('/deleteItem', async function(req, res, next) {
  try {
    res.json(await deleteData(req))
  } catch (err) {
    console.error(`Error while deleting items `, err.message);
    next(err);
  }
})

```

Aqui llamo a la sql

```

// DELETE
async function deleteData(req) {
  const data = req.query;

  const result = await db.query(
    `
    DELETE FROM coleccion
    WHERE id = ?
    `,
    [data.id]
  );

  return result.affectedRows;
}

export { insertData, getData, deleteData }

```

Se ejecuta la funcion

2025-35014664-0672 x 2025-35014664-0457 x UT2A4 - CRUD con a x Guía para backend de x localhost:3030/login? x

localhost/phpmyadmin/index.php?route=/sql&pos=0&db=bdgestion&table=coleccion

phpMyAdmin

Reciente Favoritas

Nueva bdgestion Nueva coleccion usuarios information_schema mysql performance_schema phpmyadmin test

Servidor: 127.0.0.1 Base de datos: bdgestion Tabla: coleccion

Examinar Estructura SQL Buscar Insertar Exportar Importar Privilegios

Mostrando filas 0 - 0 (total de 1, La consulta tardó 0,0002 segundos.)

SELECT * FROM `coleccion`

Perfilando [Editar en línea] [Editar] [Explicar SQL] [Crear código PHP] [Actualizar]

Mostrar todo Número de filas: 25 Filtrar filas: Buscar en esta tabla

Opciones extra

	id	nombre	marca	tipo	precio
☐ Editar Copiar Borrar	5	XDVag	retsfdfs	wrtesafgs	2

Seleccionar todo Para los elementos que están marcados: Editar Copiar Borrar Expc

Mostrar todo Número de filas: 25 Filtrar filas: Buscar en esta tabla

Se reflejan los datos en la bd